Development of a CRUD Web Application using Django REST Framework and React JS

Avinash Kumar Yadav

March 14, 2026

Abstract

Web applications are increasingly built using modern full-stack technologies that separate the frontend and backend components. This project presents the design and development of a CRUD (Create, Read, Update, Delete) web application using Django REST Framework for the backend and React JS for the frontend. The system allows users to manage student records efficiently through a web interface. The backend exposes RESTful APIs that interact with the database, while the React frontend communicates with these APIs using HTTP requests. The project demonstrates full-stack web development principles and highlights the integration of modern web frameworks for scalable application development.

Contents

1 Introduction **3**

2 Problem Statement **3**

3 Objectives **3**

4 Technology Stack **3**
4.1 Backend Technologies 3
4.2 Frontend Technologies 4

5 System Architecture **4**

6 Database Design **4**

7 Backend Implementation **4**
7.1 Model 4
7.2 Serializer 5
7.3 ViewSet 5

8 API Testing **5**

9 Backend Administration **6**

10 Frontend Implementation **6**

11 Application Interface **6**

12 Initial Application State **7**

13 Results **7**

14 GitHub Repository **8**

15 Advantages **8**

16 Future Enhancements **8**

17 Conclusion **8**

1 Introduction

With the rapid development of web technologies, modern applications increasingly rely on RESTful APIs and frontend frameworks to build scalable and maintainable systems. Traditionally, web applications were built using server-rendered templates where both frontend and backend logic were tightly coupled. However, modern development practices encourage separation of concerns, where the backend provides data through APIs and the frontend consumes these APIs to render dynamic user interfaces.

Django REST Framework (DRF) is a powerful toolkit built on top of the Django web framework. It simplifies the process of building REST APIs by providing serializers, viewsets, authentication mechanisms, and routers.

React JS is a widely used JavaScript library for building interactive user interfaces. React allows developers to create reusable components and efficiently manage application state using a virtual DOM.

This project combines Django REST Framework and React JS to implement a simple Student Management System that performs CRUD operations.

2 Problem Statement

Managing student data manually can be inefficient and error-prone. Many institutions require systems that allow easy storage, retrieval, and modification of student information. The absence of a digital system makes data management difficult and inefficient.

The goal of this project is to develop a simple web-based application that allows users to perform CRUD operations on student records through an interactive interface.

3 Objectives

The main objectives of the project are:

- To understand REST API development using Django REST Framework.
- To design a frontend user interface using React JS.
- To implement CRUD operations for managing student data.
- To integrate frontend and backend technologies using HTTP requests.
- To demonstrate full-stack web application development.

4 Technology Stack

4.1 Backend Technologies

- Python Programming Language
- Django Web Framework
- Django REST Framework

- SQLite Database
- django-cors-headers

4.2 Frontend Technologies

- React JS
- Axios
- HTML
- CSS
- JavaScript

5 System Architecture

The system follows a three-tier architecture consisting of:

- **Presentation Layer (Frontend)** – React JS user interface.
- **Application Layer (Backend)** – Django REST API handling requests.
- **Data Layer (Database)** – SQLite database storing student data.

The React application sends HTTP requests to the Django REST API. The API processes the request and communicates with the database before returning a response to the frontend.

6 Database Design

The application uses a simple database schema consisting of a single Student table.

Field	Data Type	Description heightid
Integer	Primary Key name	CharField
Student name email	EmailField	Student email address age
IntegerField	Student age height	

7 Backend Implementation

The backend is implemented using Django REST Framework.

7.1 Model

```
class Student(models.Model):
name = models.CharField(max_length=100)
email = models.EmailField()
age = models.IntegerField()
```

7.2 Serializer

Serializers convert model data into JSON format for API communication.

```
class StudentSerializer(serializers.ModelSerializer):
class Meta:
model = Student
fields = '**all**'
```

7.3 ViewSet

```
class StudentViewSet(viewsets.ModelViewSet):
queryset = Student.objects.all()
serializer_class = StudentSerializer
```

The ModelViewSet automatically provides the following API operations:

- GET – Retrieve records
- POST – Create records
- PUT – Update records
- DELETE – Remove records

8 API Testing

Django REST Framework provides a built-in interface for testing APIs.

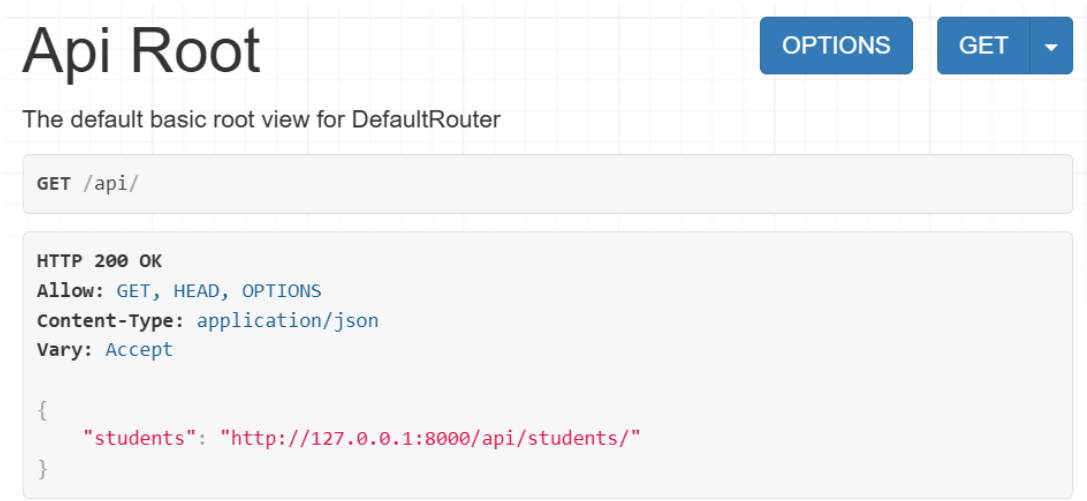

Figure 1: Django REST Framework API Root Interface

The API root page displays available endpoints for managing student records.

9 Backend Administration

Django provides a built-in administrative interface for managing database records.

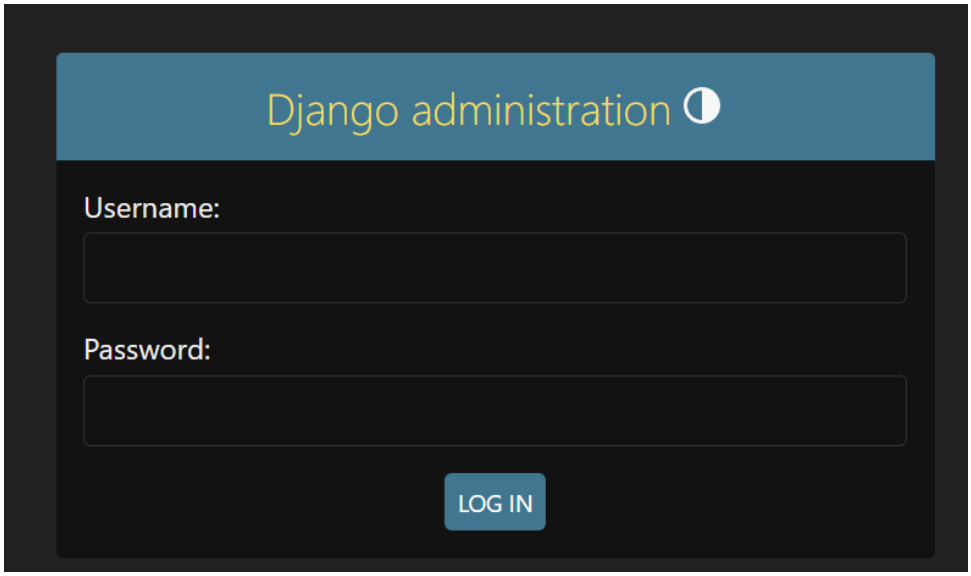

Figure 2: Django Administration Login Interface

Administrators can log in and manage student records through this interface.

10 Frontend Implementation

The frontend is implemented using React JS. React components allow users to interact with the application and perform CRUD operations.

Axios is used to send HTTP requests from React to the Django REST API.

Example configuration:

```
const API = axios.create({
baseURL: "http://127.0.0.1:8000/api/"
});
```

React components used:

- StudentForm – Add new students
- StudentList – Display student records

11 Application Interface

The user interface of the application allows users to add new students and view existing student records.

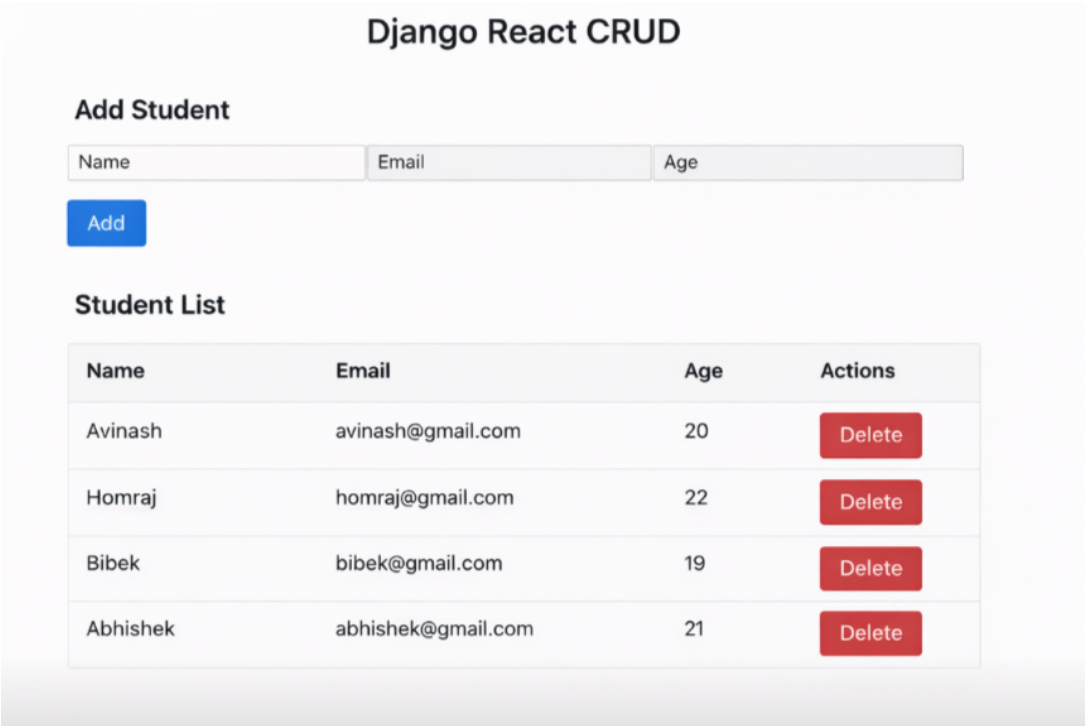

Figure 3: React Frontend Interface

12 Initial Application State

When the application loads initially, the student list is empty.

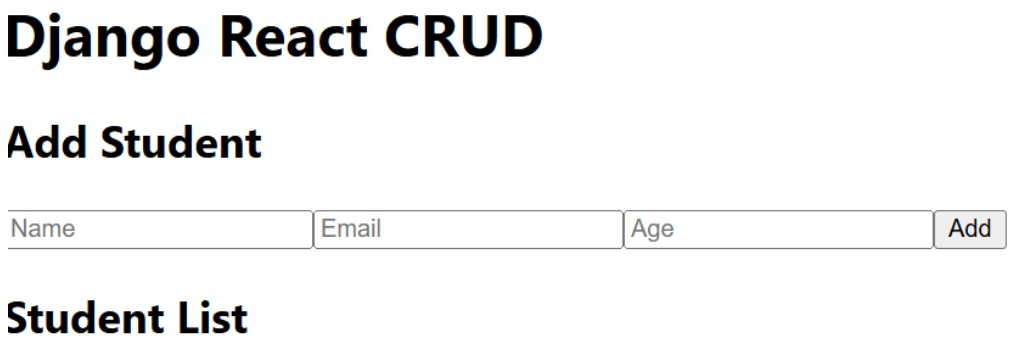

Figure 4: Initial React Interface Before Adding Records

13 Results

The developed application successfully performs CRUD operations. Users can add, view, and delete student records through the React interface.

The integration between React and Django REST Framework demonstrates the effectiveness of modern web development practices.

14 GitHub Repository

The complete source code for this project is available on GitHub.

```
https://github.com/avinashyadav17/Django-Rest-Framework-React-JS-.git
```

The repository contains both the backend implementation using Django REST Framework and the frontend implementation using React JS.

15 Advantages

- Clear separation between frontend and backend.
- REST APIs allow easy integration with other systems.
- React provides dynamic and responsive user interfaces.
- Django REST Framework simplifies backend development.

16 Future Enhancements

Future improvements may include:

- User authentication using JWT.
- Search and filtering features.
- Pagination support.
- Improved user interface using Bootstrap or Material UI.
- Deployment to cloud platforms such as Heroku or Render.

17 Conclusion

This project successfully demonstrates the development of a full-stack CRUD web application using Django REST Framework and React JS. The application allows users to manage student records efficiently through an intuitive interface.

The integration of Django REST APIs with React JS highlights the power of modern web technologies and provides a scalable architecture suitable for real-world applications.